Some Cybersecurity Lab Projects Using Python

Scenario 1

I am working as a security analyst. First, I am responsible for checking whether a user's operating system requires an update. Then, I need to investigate login attempts for a specific device. I must determine if login attempts were made by users approved to access this device and if the login attempts occurred during organization hours.

Task 1

I am asked to help automate the process of checking whether a user's operating system requires an update. According to that I need to display message. To solve this, I have stored a variable called `system.` Then I used an '`if`' statement by taking care of indentation. Then I called '`print`' function to display the message. Here, the code can be run by changing different OS, OS 1, OS 3, etc. However, at this stage, only "no update needed" message will be displayed when the OS is 2. So, I need to extend the code for next task.

```
system = "OS 2"

# If OS 2 is running, then display a "no update needed" message

if system == "OS 2":
    print("no update needed")
```

```
no update needed
```

Task 2

Now to complete the next step, I have added '`else`' statement. Python will check if the condition is not True, it will go to the next line of code. Here, Python will check if the operating system is something else other than OS 1, it will display the message "update is required".

```
system = "OS 2"

# If OS 2 is running, then display a "no update needed" message

if system == "OS 1":
    print("no update needed")
else:
    print("Update is needed.")
```

```
Update is needed.
```

Task 3

This setup is still not ideal as it will not resolve the problem. If the variable system contains a random string or integer, the conditional above would still display update needed.

To improve the conditional, I added the elif keyword. In the following cell, I added two elif statements after the if statement, to create the final code. The first elif statement would display update needed if system is "OS 1". The second elif statement would display the same message, if system is "OS 3".

```
system = "OS 1"

# If OS 2 is running, then display a "no update needed" message
# Otherwise if OS 1 is running, display a "update needed" message
# Otherwise if OS 3 is running, display a "update needed" message

if system == "OS 2":
    print("no update needed")
elif system == "OS 1":
    print("update needed")
elif system == "OS 3":
    print("update needed")
```

```
update needed
```

Task 4

I wrote the code for the same task more concisely. I used a logical operator to combine the two elif statements from the previous setup into one elif statement. Then, assigned the system variable to a value.

```
system = "OS 1"

# If OS 2 is running, then display a "no update needed" message
# Otherwise if either OS 1 or OS 3 is running, display a "update ne

if system == "OS 2":
    print("no update needed")
elif (system < "OS 2" or system > "OS 2"):
    print("update needed")
```

```
update needed
```

Task 5

Now I am moving on to the next part of my task that is to investigate login attempts to a specific device. Only approved users should log on to this device.

I started with two authorized users, stored in the variables **approved_user1** and **approved_user2**. I wrote a conditional statement that compares those variables to a third variable, **username**. This is the username of a specific user trying to log in.

```
# Assign `approved_user1` and `approved_user2` to usernames of approv

approved_user1 = "elarson"
approved_user2 = "bmoreno"

# Assign `username` to the username of a specific user trying to log

username = "beky"

# If the user trying to log in is among the approved users, then disp
# Otherwise, display a message that they do not have access to this d

if username == approved_user1 or username == approved_user2:
    print("This user has access to this device.")
else:
    print("This user does not have access to this device.")
```

```
This user does not have access to this device.
```

```
# Assign `approved_user1` and `approved_user2` to usernames of approv

approved_user1 = "elarson"
approved_user2 = "bmoreno"

# Assign `username` to the username of a specific user trying to log

username = "elarson"

# If the user trying to log in is among the approved users, then disp
# Otherwise, display a message that they do not have access to this d

if username == approved_user1 or username == approved_user2:
    print("This user has access to this device.")
else:
    print("This user does not have access to this device.")
```

```
This user has access to this device.
```

Task 6

Now, I stored the approved users in an allow list called **`approved_list`.** I have used the **`in`** operator to determine whether a specific username is part of a list of approved usernames. For example, username in **`approved_list`** evaluates to True if the value of the username variable is included in **`approved_list`.**

```
approved_list = ["elarson", "bmoreno", "tshah", "sgilmore", "eraab"]

# Assign `username` to the username of a specific user trying to log

username = "tshah"

# If the user trying to log in is among the approved users, then disp
# Otherwise, display a message that they do not have access to this d

if username in approved_list:
    print("This user has access to this device.")
else:
    print("This user does not have access to this device.")
```

```
This user has access to this device.
```

```
approved_list = ["elarson", "bmoreno", "tshah", "sgilmore", "eraab"]

# Assign `username` to the username of a specific user trying to log

username = "adam"

# If the user trying to log in is among the approved users, then disp
# Otherwise, display a message that they do not have access to this d

if username in approved_list:
    print("This user has access to this device.")
else:
    print("This user does not have access to this device.")
```

```
This user does not have access to this device.
```

Task 7

Next, I wrote another conditional statement using an **`organization_hours`** variable to check if the user logged in during specific organization hours. When that condition is met, the code displays the string "Login attempt made during organization hours.". When that condition isn't met, the code displays the string "Login attempt made outside of organization hours.".

The **`organization_hours`** variable has a Boolean value of True, that means the user is logged in during the specified organization hours. If **`organization_hours`** has a Boolean value of False, that means the user is not logged in during those hours.

```
organization_hours = True

# If the entered `organization_hours` has a value of True, then display "Login attempt made during organization hour
# Otherwise, display "Login attempt made outside of organization hours."

if organization_hours:
   print("Login attempt made during organisation hours.")
else:
   print("Login attempt made outside of organisation hours.")
```

```
Login attempt made during organisation hours.
```

Task 8

Next, I included the conditional statement that checks if a user is on the allow list and another conditional statement that checks if the user logged in during organization hours to complete my tasks.

```
approved_list = ["elarson", "bmoreno", "tshah", "sgilmore", "eraab"]

# Assign `username` to the username of a specific user trying to log in

username = "tom"

# If the user trying to log in is among the approved users, then display a message that they are approved to access
# Otherwise, display a message that they do not have access to this device

if username in approved_list:
    print("This user has access to this device.")

else:
    print("This user does not have access to this device.")

# Assign `organization_hours` to a Boolean value that represents whether the user is trying to log in during organi

organization_hours = False

# If the entered `organization_hours` has a value of True, then display "Login attempt made during organization hour
# Otherwise, display "Login attempt made outside of organization hours."

if organization_hours == True:
    print("Login attempt made during organization hours.")
else:
    print("Login attempt made outside of organization hours.")
```

```
This user does not have access to this device.
Login attempt made outside of organization hours.
```

Scenario 2

I am working as a security analyst and writing programs in Python to automate displaying messages regarding network connection attempts, detecting IP addresses that are attempting to access restricted data and generating employee ID numbers for a Sales department.

Step 1

First, I created a loop related to connecting to a network.

```
: # Used iterative statement[`for`, `range()`], and a loop variable of `i`
  # Displayed "Connection could not be established." three times

  for i in range(3):
      print("Connection could not be established.")
```

```
Connection could not be established.
Connection could not be established.
Connection could not be established.
```

Step 2

Next, I have assigned an integer value to a variable and passed that variable into the **`range()`** function within a '**`for`**' loop.

```
# Created a variable,`connection_attempts`. It stores the number of times the user has tried to connect to network

connection_attempts = 3

# Wrote iterative statement using `for`, `range()`, a loop variable of `i`, and `connection_attempts`
# Displayed "Connection could not be established." as many times as specified by `connection_attempts`

for i in range(connection_attempts):
    print("Connection could not be established")
```

```
Connection could not be established
Connection could not be established
Connection could not be established
```

Step 3

I have also used a 'for' loop and a 'while' loop to do the same task.

```
# Assigned `connection_attempts` to an initial value of 0,
# to keep track of how many times the user has tried to connect to the network

connection_attempts = 0

# Iterative statement using `while` and `connection_attempts`
# Displayed "Connection could not be established." every iteration,
# until connection_attempts reaches a specified number

while connection_attempts < 3:
    print("Connection could not be established")

    # Updated `connection_attempts` (increment it by 1 at the end of each iteration)
    connection_attempts = connection_attempts + 1
```

```
Connection could not be established
Connection could not be established
Connection could not be established
```

Step 4

After that, I moved to do the next task – automate checking whether IP addresses are part of an allow list. I have started with a list of IP addresses from which users have tried to log in, stored in a variable called **`ip_addresses`**. Then I used '`for`' loop that displayed the elements of this list one at a time. I used '**`i`**' as the loop variable in the '`for`' loop.

```
# Assigned `ip_addresses` to a list of IP addresses from which users have tried to log in

ip_addresses = ["192.168.142.245", "192.168.109.50", "192.168.86.232", "192.168.131.147",
                "192.168.205.12", "192.168.200.48"]

# For loop that displays the elements of `ip_addresses` one at a time

for i in ip_addresses:
    print(i)
```

```
192.168.142.245
192.168.109.50
192.168.86.232
192.168.131.147
192.168.205.12
192.168.200.48
```

Step 5

I am given a list of IP addresses that are allowed to log in, stored in '**`allow_list`**' variable. I wrote '`if`' statement inside of the '**`for`**' loop. For each IP address in the list of IP addresses from which users have tried to log in, display "IP address is allowed" if it is among the allowed addresses and display "IP address is not allowed" otherwise.

```
# Assign `allow_list` to a list of IP addresses that are allowed to log in

allow_list = ["192.168.243.140", "192.168.205.12", "192.168.151.162", "192.168.178.71",
              "192.168.86.232", "192.168.3.24", "192.168.170.243", "192.168.119.173"]

# Assign `ip_addresses` to a list of IP addresses from which users have tried to log in

ip_addresses = ["192.168.142.245", "192.168.109.50", "192.168.86.232", "192.168.131.147",
                "192.168.205.12", "192.168.200.48"]

# For each IP address in the list of IP addresses from which users have tried to log in,
# If it is among the allowed addresses, then display “IP address is allowed”
# Otherwise, display “IP address is not allowed”

for i in ip_addresses:
    if i in allow_list:
        print("IP address is allowed")
    else:
        print("IP address is not allowed")
```

```
IP address is not allowed
IP address is not allowed
IP address is allowed
IP address is not allowed
IP address is allowed
IP address is not allowed
```

Step 6

Here if any user outside the allow list tries to access any restricted information, the system will display a message about further actions. I have used ‘break’ keyword and expanded the message.

```
# Assign `allow_list` to a list of IP addresses that are allowed to log in

allow_list = ["192.168.243.140", "192.168.205.12", "192.168.151.162", "192.168.178.71",
              "192.168.86.232", "192.168.3.24", "192.168.170.243", "192.168.119.173"]

# Assign `ip_addresses` to a list of IP addresses from which users have tried to log in

ip_addresses = ["192.168.142.245", "192.168.109.50", "192.168.86.232", "192.168.131.147",
                "192.168.205.12", "192.168.200.48"]

# For each IP address in the list of IP addresses from which users have tried to log in,
# If it is among the allowed addresses, then display “IP address is allowed”
# Otherwise, display “IP address is not allowed”

for i in ip_addresses:
    if i in allow_list:
        print("IP address is allowed")
    else:
        print("IP address is not allowed. Further investigation of login activity required")
        break
```

```
IP address is not allowed. Further investigation of login activity required
```

Task 7

Next, I have automated the creation of new employees by using conditional '`while`' loop. This creates unique employee IDs for the Sales department by iterating through numbers and displays each ID created.

```
# Assign the loop variable `i` to an initial value of 5000

i = 5000

# While loop that generates unique employee IDs for the Sales department by iterating through numbers
# and displays each ID created

while i <= 5150:
    print(i)
    i = i + 5
```

Step 8

Finally, I want to incorporate a message that displays 'Only 10 valid employee ids remaining' as a helpful alert once the loop variable reaches to 5100. I have included '`if`' statement.

```
# Assign the loop variable `i` to an initial value of 5000

i = 5000

# While loop that generates unique employee IDs for the Sales department by iterating through numbers
# and displays each ID created
# This loop displays "Only 10 valid employee ids remaining" once `i` reaches 5100

while i <= 5150:
    print(i)
    if i == 5100:
        print("Only 10 valid employee ids remaining")
    i = i + 5
```

This is how I have completed scenario 2 task.

Scenario 3

Here, I am asked to define and call a function that displays an alert about potential security issue. Also, I will be working with a list of employee usernames, creating a function that converts the list into one string.

First, I have created a function call '`alert()`' and called it.

```
# Define a function named `alert()`

def alert():
    print("Potential security issue. Investigate further.")

# Call the `alert()` function

alert()
```

```
Potential security issue. Investigate further.
```

Next, the organisation provided me a list of approved usernames, stored in a variable named '**`username_list`**'. My task is to show each element from the **`username_list`** on a new line.

```
# Define a function named `list_to_string()`

def list_to_string():

  # Store the list of approved usernames in a variable named `username_list`

  username_list = ["elarson", "bmoreno", "tshah", "sgilmore", "eraab", "gesparza", "alevitsk", "wjaffrey"]

  # Write a for loop that iterates through the elements of `username_list` and displays each element

  for i in username_list:
    print(i)

# Call the `list_to_string()` function

list_to_string()
```

```
elarson
bmoreno
tshah
sgilmore
eraab
gesparza
alevitsk
wjaffrey
```

Finally, I have modified the code to call the function in a string to finish scenario 3 task.

```
# Define a function named `list_to_string()`

def list_to_string():

  # Store the list of approved usernames in a variable named `username_list`

  username_list = ["elarson", "bmoreno", "tshah", "sgilmore", "eraab", "gesparza", "alevitsk", "wjaffrey"]

  # Assign `sum_variable` to an empty string

  sum_variable = ""

  # Write a for loop that iterates through the elements of `username_list` and displays each element

  for i in username_list:
    sum_variable = sum_variable + i + ", "

  # Display the value of `sum_variable`

  print(sum_variable)

# Call the `list_to_string()` function

list_to_string()
```

```
elarson, bmoreno, tshah, sgilmore, eraab, gesparza, alevitsk, wjaffrey,
```

Scenario 4

I am working as a security analyst; I am responsible for working with a list that contains the number of failed logins attempts that occur each month. I need to identify any patterns that might indicate malicious activity. I am also responsible for defining a function that compares the logins for the current day to an average and improving it by a return statement for future use.

Here is how I have done the above task:

First, I have sorted the given failed login list. This allows me to see that the last two numbers in the sorted list (223, 264) are outlying numbers, indicating an increase in the number of failed login attempts. Then used `max()` function to isolate the highest number of login attempts (264).

```
failed_login_list = [119, 101, 99, 91, 92, 105, 108, 85, 88, 90, 264, 223]

print(sorted(failed_login_list))
print(max(failed_login_list))
```

```
[85, 88, 90, 91, 92, 99, 101, 105, 108, 119, 223, 264]
264
```

Next, I have defined a function called analyze_logins with three parameters (`username, current_day_logins, average_day_logins`) and wrote some necessary messages based on the parameters. Also, I have created a variable (`login_ratio`) to store the ratio of the logins made on the

current day to the logins made on an average day. After that, I have used 'return' keyword to store that information for future usage. Finally, I wrote a message to alert the security team if the login attempt is more than or equal to 3. In this case, this should be investigated further, means need attention.

```
# Define a function named `analyze_logins()` that takes in three parameters, `username`, `current_day_logins`, and `
def analyze_logins(username, current_day_logins, average_day_logins):
    # Display a message about how many login attempts the user has made that day
    print("Current day login total for", username, "is", current_day_logins)
    # Display a message about average number of login attempts the user has made that day
    print("Average logins per day for", username, "is", average_day_logins)
    # Calculate the ratio of the logins made on the current day to the logins made on an average day, storing in a v
    login_ratio = current_day_logins / average_day_logins
    # Return the ratio
    return login_ratio
# Call `analyze_logins() and store the output in a variable named `login_analysis`
login_analysis = analyze_logins("ejones", 9, 3)
# Conditional statement that displays an alert about the login activity if it's more than normal
if login_analysis >= 3:
    print("Alert! This account has more login activity than normal.")
```

```
Current day login total for ejones is 9
Average logins per day for ejones is 3
Alert! This account has more login activity than normal.
```

Scenario 5

As a security analyst, first, I am asked to update employee ID where it is less than 5 digits to a given alphanumerical format. Then, I am asked to extract URL information from a given string.

To do the first task, I have converted the integer into the string, wrote the conditional statement, used concatenation and added the given instruction to the existing format. This is how I have updated the employee ID according to the organisational need.

```
# Assign `employee_id` to a four digit number as an initial value

employee_id = 4186

# Reassign `employee_id` to the same value but in the form of a string

employee_id = str(employee_id)

# Display the `employee_id` as it currently stands

print(employee_id)

# Conditional statement that updates the `employee_id` if its length is less than 5 digits

if len(employee_id) < 5:
    employee_id = "E" + employee_id

# Display the `employee_id` after the update

print(employee_id)
```

```
4186
E4186
```

For the second task, extracting URL, I have stored the given string in a variable, applied index method to another new variable and printed the output using Python's string slice method.

```
# Assign `url` to a specific URL

url = "https://exampleURL1.com"

# Assign `ind` to the output of applying `.index()` to `url` in order to extract the starting index of ".com" in `url

ind = url.index(".com")

# Extract the website name in `url` and display it

print(url[8:ind])
```

```
exampleURL1
```

So, I have updated the employee ID into the right format and extracted the URL name with the help of Python.

Developing algorithm lab

Scenario 6

In this lab, as a security analyst, I am responsible for developing an algorithm that connects users to their assigned devices. When any employee will leave, the username and assigned device will be updated. Also, any new user will be added to the user list along with the assigned device. I am asked to write code that indicates if a user is approved on the system and has brought their assigned device to the security team.

First, I wrote code to add any new user with allocated device for them.

```
# Assign `approved_users` to a list of approved usernames

approved_users = ["elarson", "bmoreno", "tshah", "sgilmore", "eraab"]

# Assign `approved_devices` to a list of device IDs that correspond to the usernames in `appr

approved_devices = ["8rp2k75", "hl0s5o1", "2ye3lzg", "4n482ts", "a307vir"]

# Assign `new_user` to the username of a new approved user

new_user = "gesparza"

# Assign `new_device` to the device ID of the new approved user

new_device = "3rcv4w6"

# Add that user's username and device ID to `approved_users` and `approved_devices` respectiv

approved_users.append(new_user)
approved_devices.append(new_device)

# Display the contents of `approved_users`

print(approved_users)

# Diplay the contents of `approved_devices`

print(approved_devices)
```

```
['elarson', 'bmoreno', 'tshah', 'sgilmore', 'eraab', 'gesparza']
['8rp2k75', 'hl0s5o1', '2ye3lzg', '4n482ts', 'a307vir', '3rcv4w6']
```

Next, I have modified the code to remove any employee and allocated device from the system when they leave the company.

```
# Assign `approved_users` to a list of approved usernames

approved_users = ["elarson", "bmoreno", "tshah", "sgilmore", "eraab", "gesparza"]

# Assign `approved_devices` to a list of device IDs that correspond to the usernames in `appr

approved_devices = ["8rp2k75", "hl0s5o1", "2ye3lzg", "4n482ts", "a307vir", "3rcv4w6"]

# Assign `removed_user` to the username of the employee who has left the team

removed_user = "tshah"

# Assign `removed_device` to the device ID of the employee who has left the team

removed_device = "2ye3lzg"

# Remove that employee's username and device ID from `approved_users` and `approved_devices`

approved_users.remove(removed_user)
approved_devices.remove(removed_device)

# Display `approved_users`

print(approved_users)

# Diplay `approved_devices`

print(approved_devices)
```

```
['elarson', 'bmoreno', 'sgilmore', 'eraab', 'gesparza']
['8rp2k75', 'hl0s5o1', '4n482ts', 'a307vir', '3rcv4w6']
```

After that, I have created an algorithm to determine if a username and device ID correspond, I have written a conditional statement with message for approved user and device. The user will receive a negative message if they are not approved or their device is incorrect.

```
# Assign `approved_users` to a list of approved usernames

approved_users = ["elarson", "bmoreno", "sgilmore", "eraab", "gesparza"]

# Assign `approved_devices` to a list of device IDs that correspond to the usernames in `approved_users`

approved_devices = ["8rp2k75", "hl0s5o1", "4n482ts", "a307vir", "3rcv4w6"]

# Assign `username` to a username

username = "sgilmore"

# Assign `device_id` to a device ID

device_id = "4n482ts"

# Assign `ind` to the index of `username` in `approved_users`

ind = approved_users.index(username)

# If statement
# If `username` belongs to `approved_users`, and if the element at `ind` in `approved_devices` matches `device_id`,
# then display a message that the username is approved,
# followed by a message that the user has the correct device

if username in approved_users and device_id == approved_devices[ind]:
    print("The user", username, "is approved to access the system.")
    print(device_id, "is the assigned device for", username)

# Elif statement
# Handles the case when `username` belongs to `approved_users` but element at `ind` in `approved_devices` does not ma
# and displays two messages accordingly

elif username in ind != approved_devices:
    print("The user", username, "is approved to access the system, but", device_id, "is not their assigned device.")
else:
    print("The user is not an approved user and does not have any assigned device.")
```

```
The user sgilmore is approved to access the system.
4n482ts is the assigned device for sgilmore
```

Finally, I have completed the algorithm by defining a function (login) with two parameters (username and device) and called them.

```
approved_devices = ["8rp2k75", "hl0s5o1", "4n482ts", "a307vir", "3rcv4w6"]

def login(username, device_id):

    if username in approved_users:

        print("The user", username, "is approved to access the system.")

        ind = approved_users.index(username)

        if device_id == approved_devices[ind]:

            print(device_id, "is the assigned device for", username)

        else:

            print(device_id, "is not their assigned device.")

    # Otherwise (part of the outer conditional and handles the case when `username` does not belong to `approved_user

    else:

        print("The username", username, "is not approved to access the system.")

login("elarson", "8rp2k75")
print("  " * 20)

login("eraab", "a307vir")
print("  " * 20)

login("bmoreno", "5tf389")
```

```
The user elarson is approved to access the system.
8rp2k75 is the assigned device for elarson

The user eraab is approved to access the system.
a307vir is the assigned device for eraab

The user bmoreno is approved to access the system.
5tf389 is not their assigned device.
```

Scenario 7

In this lab, I am a security analyst, and my tasks are as follows:

- Extracting device IDs containing certain characters from a log; these characters correspond with a certain operating system that requires an update.
- Extracting all IP addresses from a log and then comparing them to those that are flagged in a list.

Step 1

I am asked to look for device IDs that begin with '**r15**'. To do this, I have imported '**re**' module, stored all the devices in a variable (devices), and displayed their names.

```
import re
# Assign `devices` to a string containing device IDs, each device ID represented by alphanumeric characters

devices = "r262c36 67bv8fy 41j1u2e r151dm4 1270t3o 42dr56i r15xk9h 2j33krk 253be78 ac742a1 r15u9q5 zh86b2l ii286fq 9x

# Display the contents of `devices`
print(devices)
```

```
r262c36 67bv8fy 41j1u2e r151dm4 1270t3o 42dr56i r15xk9h 2j33krk 253be78 ac742a1 r15u9q5 zh86b2l ii286fq 9x482kt 6oa6
m6u x3463ac i4l56nq g07h55q 081qc9t r159r1u
```

Then, I wrote a pattern to find devices that start with the character combination of '**r15**'. Here, I have used '**\w**' and '+' to create a pattern and store it as a string in **'target_pattern'** variable. The symbol '**\w**' will find any alphanumeric value and '**+**' will find matched string with any length. After that, I have used **findall()** function from re module.

```
import re
# Assign `devices` to a string containing device IDs, each device ID represented by alphanumeric characters

devices = "r262c36 67bv8fy 41j1u2e r151dm4 1270t3o 42dr56i r15xk9h 2j33krk 253be78 ac742a1 r15u9q5 zh86b2l ii286fq 9x

# Assign `target_pattern` to a regular expression pattern for finding device IDs that start with "r15"

target_pattern = "r15\w+"

# Use `re.findall()` to find the device IDs that start with "r15" and display the results

print(re.findall(target_pattern, devices))
```

```
['r151dm4', 'r15xk9h', 'r15u9q5', 'r159r1u']
```

My next task is to analyse a network security log file and determine which IP addresses have been flagged for unusual activity. To do this, first I have stored the log file in a variable (log_file) and displayed it.

```
# Assign `log_file` to a string containing username, date, login time, and IP address for a series of login attempts

log_file = "eraab 2022-05-10 6:03:41 192.168.152.148 \niuduike 2022-05-09 6:46:40 192.168.22.115 \nsmartell 2022-05-0

# Display contents of `log_file`

print(log_file)
```

```
eraab 2022-05-10 6:03:41 192.168.152.148
iuduike 2022-05-09 6:46:40 192.168.22.115
smartell 2022-05-09 19:30:32 192.168.190.178
arutley 2022-05-12 17:00:59 1923.1689.3.24
rjensen 2022-05-11 0:59:26 192.168.213.128
aestrada 2022-05-09 19:28:12 1924.1680.27.57
asundara 2022-05-11 18:38:07 192.168.96.200
dkot 2022-05-12 10:52:00 1921.168.1283.75
abernard 2022-05-12 23:38:46 19245.168.2345.49
cjackson 2022-05-12 19:36:42 192.168.247.153
jclark 2022-05-10 10:48:02 192.168.174.117
alevitsk 2022-05-08 12:09:10 192.16874.1390.176
jrafael 2022-05-10 22:40:01 192.168.148.115
yappiah 2022-05-12 10:37:22 192.168.103.10654
daquino 2022-05-08 7:02:35 192.168.168.144
```

Then I built a regular expression pattern that I can use later to extract IP addresses that are in the form of xxx.xxx.xxx.xxx. I have used symbol '**\d**' (to obtain digits) and '**\.**' for period.

```
# Assign `log_file` to a string containing username, date, login time, and IP address for a series of login attempts

log_file = "eraab 2022-05-10 6:03:41 192.168.152.148 \niuduike 2022-05-09 6:46:40 192.168.22.115 \nsmartell 2022-05-0

# Assign `pattern` to a regular expression pattern that will match with IP addresses of the form xxx.xxx.xxx.xxx

pattern = "\d\d\d\."
print(re.findall(pattern, log_file))
```

```
['192.', '168.', '152.', '192.', '168.', '192.', '168.', '190.', '923.', '689.', '192.', '168.', '213.', '924.', '68
0.', '192.', '168.', '921.', '168.', '283.', '245.', '168.', '345.', '192.', '168.', '247.', '192.', '168.', '174.',
'192.', '874.', '390.', '192.', '168.', '148.', '192.', '168.', '103.', '192.', '168.', '168.']
```

Next, I used the '**`re.findall()`**' function on the regular expression pattern stored in '**`pattern`**' variable and the provided log file to extract the corresponding IP addresses. Here, noticeable that all IP addresses are not obtained.

```
# Assign `log_file` to a string containing username, date, login time, and IP address for a series of login attempts

log_file = "eraab 2022-05-10 6:03:41 192.168.152.148 \niuduike 2022-05-09 6:46:40 192.168.22.115 \nsmartell 2022-05-0
print(log_file)

# Assign `pattern` to a regular expression pattern that will match with IP addresses of the form xxx.xxx.xxx.xxx

pattern = "\d\d\d\.\d\d\d\.\d\d\d\.\d\d\d"

# Use the `re.findall()` function on `pattern` and `log_file` to extract the IP addresses of the form xxx.xxx.xxx.xxx

print(re.findall(pattern, log_file))
```

```
eraab 2022-05-10 6:03:41 192.168.152.148
iuduike 2022-05-09 6:46:40 192.168.22.115
smartell 2022-05-09 19:30:32 192.168.190.178
arutley 2022-05-12 17:00:59 1923.1689.3.24
rjensen 2022-05-11 0:59:26 192.168.213.128
aestrada 2022-05-09 19:28:12 1924.1680.27.57
asundara 2022-05-11 18:38:07 192.168.96.200
dkot 2022-05-12 10:52:00 1921.168.1283.75
abernard 2022-05-12 23:38:46 19245.168.2345.49
cjackson 2022-05-12 19:36:42 192.168.247.153
jclark 2022-05-10 10:48:02 192.168.174.117
alevitsk 2022-05-08 12:09:10 192.16874.1390.176
jrafael 2022-05-10 22:40:01 192.168.148.115
yappiah 2022-05-12 10:37:22 192.168.103.10654
daquino 2022-05-08 7:02:35 192.168.168.144
['192.168.152.148', '192.168.190.178', '192.168.213.128', '192.168.247.153', '192.168.174.117', '192.168.148.115',
'192.168.103.106', '192.168.168.144']
```

Now, there are some IP addresses in the log file that are not extracted yet because of each segment of digits in a valid IP address (1-3). To solve this, I have used '**`+`**' symbol after '**`\d`**'.

```
# Assign `log_file` to a string containing username, date, login time, and IP address for a series of login attempts

log_file = "eraab 2022-05-10 6:03:41 192.168.152.148 \niuduike 2022-05-09 6:46:40 192.168.22.115 \nsmartell 2022-05-0

# Update `pattern` to a regular expression pattern that will match with IP addresses with any variation in the number

pattern = "\d+"

# Use the `re.findall()` function on `pattern` and `log_file` to extract the IP addresses of the updated form specife

print(re.findall(pattern, log_file))
```

```
['2022', '05', '10', '6', '03', '41', '192', '168', '152', '148', '2022', '05', '09', '6', '46', '40', '192', '168',
'22', '115', '2022', '05', '09', '19', '30', '32', '192', '168', '190', '178', '2022', '05', '12', '17', '00', '59',
'1923', '1689', '3', '24', '2022', '05', '11', '0', '59', '26', '192', '168', '213', '128', '2022', '05', '09', '1
9', '28', '12', '1924', '1680', '27', '57', '2022', '05', '11', '18', '38', '07', '192', '168', '96', '200', '2022',
'05', '12', '10', '52', '00', '1921', '168', '1283', '75', '2022', '05', '12', '23', '38', '46', '19245', '168', '23
45', '49', '2022', '05', '12', '19', '36', '42', '192', '168', '247', '153', '2022', '05', '10', '10', '48', '02',
'192', '168', '174', '117', '2022', '05', '08', '12', '09', '10', '192', '16874', '1390', '176', '2022', '05', '10',
'22', '40', '01', '192', '168', '148', '115', '2022', '05', '12', '10', '37', '22', '192', '168', '103', '10654', '2
022', '05', '08', '7', '02', '35', '192', '168', '168', '144']
```

*At this stage, Ip addresses are extracted. However, there are some invalid IP addresses with more than three digits per segment. To solve this, I have used '**`\d{1,3}\.`**' where '\d' will find digits, '**`{1,3}`**' will find any occurrences between 1 and 3 and '**`\.`**' will be for period.*

```
# Assign `log_file` to a string containing username, date, login time, and IP address for a series of login attempts

log_file = "eraab 2022-05-10 6:03:41 192.168.152.148 \niuduike 2022-05-09 6:46:40 192.168.22.115 \nsmartell 2022-05-0

# Assign `pattern` to a regular expression that matches with all valid IP addresses and only those

pattern = "\d{1,3}\."

# Use `re.findall()` on `pattern` and `log_file` and assign `valid_ip_addresses` to the output

valid_ip_addresses = re.findall(pattern, log_file)

# Display the contents of `valid_ip_addresses`

print(valid_ip_addresses)
```

```
['192.', '168.', '152.', '192.', '168.', '22.', '192.', '168.', '190.', '923.', '689.', '3.', '192.', '168.', '21
3.', '924.', '680.', '27.', '192.', '168.', '96.', '921.', '168.', '283.', '245.', '168.', '345.', '192.', '168.',
'247.', '192.', '168.', '174.', '192.', '874.', '390.', '192.', '168.', '148.', '192.', '168.', '103.', '192.', '16
8.', '168.']
```

*After that, I have stored the flagged IP addresses in a variable (**`flagged_addresses`**) and displayed them.*

```
# Assign `flagged_addresses` to a list of IP addresses that have been previously flagged for unusual activity

flagged_addresses = ["192.168.190.178", "192.168.96.200", "192.168.174.117", "192.168.168.144"]

# Display the contents of `flagged_addresses`

print(flagged_addresses)
```

```
['192.168.190.178', '192.168.96.200', '192.168.174.117', '192.168.168.144']
```

Finally, I wrote an iterative statement that loops through the **`valid_ip_addresses`** *list and checks if each IP address is flagged. I have also used conditional that checked if the address belongs to the* **`flagged_addresses`** *list. If so, necessary message is displayed, and if not, it displayed that no further analysis is required.*

```
# Assign `log_file` to a string containing username, date, login time, and IP address for a series of login attempts

log_file = "eraab 2022-05-10 6:03:41 192.168.152.148 \niuduike 2022-05-09 6:46:40 192.168.22.115 \nsmartell 2022-05-0

# Assign `pattern` to a regular expression that matches with all valid IP addresses and only those

pattern = "\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}"

# Use `re.findall()` on `pattern` and `log_file` and assign `valid_ip_addresses` to the output

valid_ip_addresses = re.findall(pattern, log_file)

# Assign `flagged_addresses` to a list of IP addresses that have been previously flagged for unusual activity

flagged_addresses = ["192.168.190.178", "192.168.96.200", "192.168.174.117", "192.168.168.144"]


for address in valid_ip_addresses:

    # If `address` belongs to `flagged_addresses`, display "The IP address ______ has been flagged for further analys

    if address in flagged_addresses:
        print("The IP address", address, "has been flagged for further analysis.")

    # Otherwise, display "The IP address ______ does not require further analysis."

    else:
        print("The IP address ", address, "does not require further analysis.")
```

```
The IP address  192.168.152.148 does not require further analysis.
The IP address  192.168.22.115 does not require further analysis.
The IP address 192.168.190.178 has been flagged for further analysis.
The IP address  192.168.213.128 does not require further analysis.
The IP address 192.168.96.200 has been flagged for further analysis.
The IP address  192.168.247.153 does not require further analysis.
The IP address 192.168.174.117 has been flagged for further analysis.
The IP address  192.168.148.115 does not require further analysis.
The IP address  192.168.103.106 does not require further analysis.
The IP address 192.168.168.144 has been flagged for further analysis.
```

From the display, it is clearly visible that which IP address is flagged and which one is not.

Scenario 8

As a security analyst, I am responsible for preparing a security log file for analysis and creating a text file with IP addresses that are allowed to access restricted information.

To do the task, first, I imported the logfile, used **`read()`** method to read the imported file, and stored that in '**`text`**' variable.

```
import_file = "data/login.txt"


with open(import_file, "r") as file:

    text = file.read()

print(text)
```

```
username,ip_address,time,date
tshah,192.168.92.147,15:26:08,2022-05-10
dtanaka,192.168.98.221,9:45:18,2022-05-09
tmitchel,192.168.110.131,14:13:41,2022-05-11
daquino,192.168.168.144,7:02:35,2022-05-08
eraab,192.168.170.243,1:45:14,2022-05-11
jlansky,192.168.238.42,1:07:11,2022-05-11
acook,192.168.52.90,9:56:48,2022-05-10
asundara,192.168.58.217,23:17:52,2022-05-12
jclark,192.168.214.49,20:49:00,2022-05-10
cjackson,192.168.247.153,19:36:42,2022-05-12
jclark,192.168.197.247,14:11:04,2022-05-12
apatel,192.168.46.207,17:39:42,2022-05-10
mabadi,192.168.96.244,10:24:43,2022-05-12
iuduike,192.168.131.147,17:50:00,2022-05-11
abellmas,192.168.60.111,13:37:05,2022-05-10
gesparza,192.168.148.80,6:30:14,2022-05-11
cgriffin,192.168.4.157,23:04:05,2022-05-09
alevitsk,192.168.210.228,8:10:43,2022-05-08
eraab,192.168.24.12,11:29:27,2022-05-11
jsoto,192.168.25.60,5:09:21,2022-05-09
```

Next, I have used '**`.split()`**' to get the result into a list of string where every line is one string.

```
import_file = "data/login.txt"

with open(import_file, "r") as file:

    text = file.read()

print(text.split())
```

```
['username,ip_address,time,date', 'tshah,192.168.92.147,15:26:08,2022-05-10', 'dtanaka,192.1
68.98.221,9:45:18,2022-05-09', 'tmitchel,192.168.110.131,14:13:41,2022-05-11', 'daquino,192.
168.168.144,7:02:35,2022-05-08', 'eraab,192.168.170.243,1:45:14,2022-05-11', 'jlansky,192.16
8.238.42,1:07:11,2022-05-11', 'acook,192.168.52.90,9:56:48,2022-05-10', 'asundara,192.168.5
8.217,23:17:52,2022-05-12', 'jclark,192.168.214.49,20:49:00,2022-05-10', 'cjackson,192.168.2
47.153,19:36:42,2022-05-12', 'jclark,192.168.197.247,14:11:04,2022-05-12', 'apatel,192.168.4
6.207,17:39:42,2022-05-10', 'mabadi,192.168.96.244,10:24:43,2022-05-12', 'iuduike,192.168.13
1.147,17:50:00,2022-05-11', 'abellmas,192.168.60.111,13:37:05,2022-05-10', 'gesparza,192.16
8.148.80,6:30:14,2022-05-11', 'cgriffin,192.168.4.157,23:04:05,2022-05-09', 'alevitsk,192.16
8.210.228,8:10:43,2022-05-08', 'eraab,192.168.24.12,11:29:27,2022-05-11', 'jsoto,192.168.25.
60,5:09:21,2022-05-09']
```

Next, there is a missing entry in the log file. I'll need to account for that by appending it the log file. I have been given missing entry. For this, I have stored those in a **`missing_entry`** variable. Then, I wrote **`open()`** function using **`write()`** method with a parameter '**a**'. Then, I used another '**`.read()`**' method to read the updated file into the '**`text`**' variable and displayed it.

```
import_file = "data/login.txt"

missing_entry = "jrafael,192.168.243.140,4:56:27,2022-05-09"

with open(import_file, "a") as file:

    file.write(missing_entry)

with open(import_file, "r") as file:

    text = file.read()

print(text)
```

```
username,ip_address,time,date
tshah,192.168.92.147,15:26:08,2022-05-10
dtanaka,192.168.98.221,9:45:18,2022-05-09
tmitchel,192.168.110.131,14:13:41,2022-05-11
daquino,192.168.168.144,7:02:35,2022-05-08
eraab,192.168.170.243,1:45:14,2022-05-11
jlansky,192.168.238.42,1:07:11,2022-05-11
acook,192.168.52.90,9:56:48,2022-05-10
asundara,192.168.58.217,23:17:52,2022-05-12
jclark,192.168.214.49,20:49:00,2022-05-10
cjackson,192.168.247.153,19:36:42,2022-05-12
jclark,192.168.197.247,14:11:04,2022-05-12
apatel,192.168.46.207,17:39:42,2022-05-10
mabadi,192.168.96.244,10:24:43,2022-05-12
iuduike,192.168.131.147,17:50:00,2022-05-11
abellmas,192.168.60.111,13:37:05,2022-05-10
gesparza,192.168.148.80,6:30:14,2022-05-11
cgriffin,192.168.4.157,23:04:05,2022-05-09
alevitsk,192.168.210.228,8:10:43,2022-05-08
eraab,192.168.24.12,11:29:27,2022-05-11
jsoto,192.168.25.60,5:09:21,2022-05-09
jrafael,192.168.243.140,4:56:27,2022-05-09
```

Next, I am asked to create a text file that consists of a list of IP addresses that are allowed to access restricted information. So, I started with creating a variable '**`import_file`**' that stores the '**allow_list.txt**' file. Then, I wrote '**with**' statement in order to write the IP addresses to the text file that I created already. After that, I wrote another '**with**' statement that reads the text file and stores it in a new variable '**text**'.

```
import_file = "data/allow_list.txt"

ip_addresses = "192.168.218.160 192.168.97.225 192.168.145.158 192.168.108.13 192.168.60.153


with open(import_file, "w") as file:

    file.write(ip_addresses)


with open(import_file, "r") as file:

    text = file.read()

print(text)
```

```
192.168.218.160 192.168.97.225 192.168.145.158 192.168.108.13 192.168.60.153 192.168.96.200
192.168.247.153 192.168.3.252 192.168.116.187 192.168.15.110 192.168.39.246
```

This is how I have completed the task. In summary, I can import and parse files to analyse the security logs.

Scenario 9

In this scenario, my task as a security analyst is to develop an algorithm that parses a file containing IP addresses that are allowed to access restricted content and removes addresses that no longer have access. I completed this task step by step.

In first step, I stored the given information into two variables (**`import_file`** and **`remove_list`**) and displayed them.

Next step, I wrote a '**`with`**' statement with **`open()`** function along with a "**r**" parameter. I then stored it in a new variable '**`ip_addresses`**' and displayed it. Then I convert that from string to list by using '**`.split()`**' method. Next, I wrote '**`for`**' loop with conditional statement that removes the elements of '**`remove_list`**' from the '**`ip_addresses`**'. After this, I wrote another '**`with`**' statement to verify that the original file was rewritten using the correct list.

```
import_file = "allow_list.txt"
remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]


with open(import_file, "r") as file:

    ip_addresses = file.read()

ip_addresses = ip_addresses.split()

for element in ip_addresses:

    if element in remove_list:

        ip_addresses.remove(element)

ip_addresses = " ".join(ip_addresses)


with open(import_file, "w") as file:

  # Rewrote the file, replacing its contents with `ip_addresses`

  file.write(ip_addresses)

# Wrote `with` statement to read in the updated file

with open(import_file, "r") as file:

    # Read in the updated file and store the contents in `text`

    text = file.read()

print(text)
```

```
ip_address 192.168.25.60 192.168.205.12 192.168.6.9 192.168.52.90 192.168.90.124 192.168.18
6.176 192.168.133.188 192.168.203.198 192.168.218.219 192.168.52.37 192.168.156.224 192.168.
60.153 192.168.69.116
```

Then, I created a function called **`update_file`** with two parameters **(`import_file`** and **`remove_list`**) and adjusted the code to complete the function of **`update_file`**.

```
# Defined a function named `update_file` that takes in two parameters: `import_file` and `remove_list`
def update_file(import_file, remove_list):
  # Wrote `with` statement to read in the initial contents of the file
  with open(import_file, "r") as file:
    # Used `.read()` to read the imported file and store it in a variable named `ip_addresses`
    ip_addresses = file.read()
  # Used `.split()` to convert `ip_addresses` from a string to a list
  ip_addresses = ip_addresses.split()
for element in ip_addresses:
    if element in remove_list:
        ip_addresses.remove(element)
  # Converted `ip_addresses` back to a string so that it can be written into the text file
  ip_addresses = " ".join(ip_addresses)
  # wrote `with` statement to rewrite the original file
  with open(import_file, "w") as file:
    # Rewrote the file, replacing its contents with `ip_addresses`
    file.write(ip_addresses)
# Called `update_file()` and passed in "allow_list.txt" and a list of IP addresses to be removed
    print(update_file, "allow_list.txt")
# Wrote `with` statement to read in the updated file
with open("allow_list.txt", "r") as file:
  # Read in the updated file and store the contents in `text`
  text = file.read()
# Displayed the contents of `text`
print(text)
```

```
ip_address 192.168.25.60 192.168.205.12 192.168.6.9 192.168.52.90 192.168.90.124 192.168.186.176 192.168.133.188 19
2.168.203.198 192.168.218.219 192.168.52.37 192.168.156.224 192.168.60.153 192.168.69.116
```

By the above code, I have parsed file containing IP addresses that are allowed to access restricted content and removes addresses that no longer have access.

Update a file through a Python algorithm

Project description

At my organisation, access to restricted content is controlled with an allow list of IP addresses. The **"allow_list.txt"** file identifies these addresses. A separate remove list identifies IP addresses that should no longer have access to this content. I created an algorithm to automate updating the "**allow_list.txt**" file and remove these IP addresses that should no longer have access.

Open the file that contains the allow list

For the first part of the algorithm, I opened the "**allow_list.txt**" file. First, I assigned this file name as a string to the **import_file** variable:

```
# Assign `import_file` to the name of the file

import_file = "allow_list.txt"
```

Then, I used a **with** statement to open the file:

```
# Build `with` statement to read in the initial contents of the file

with open(import_file, "r") as file:
```

In my algorithm, the **with** statement is used with the **.open()** function in read mode to open the allow list file for reading purpose. The reason of opening the file is to allow me to access the IP addresses stored in the allow list file. The **with** keyword will help to manage the resources by closing the file after exiting the **with** statement. In the code **with open(import_file, "r") as file:,** the **open()** function has two parameters. The first identifies the file to import, and then the second indicates what I want to do with the file. In this case, "**r**" indicates that I want to read it. The code also uses the as keyword to assign a variable named file; file stores the output of the **.open()** function while I work within the with statement.

Read the file contents

In order to read the file contents, I used the **.read()** method to convert it into the string.

```
with open(import_file, "r") as file:

  # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`

  ip_addresses = file.read()
```

When using an **`.open()`** function that includes the argument "**`r`**" for "**`read`**," I can call the **`.read()`** function in the body of the with statement. The **`.read()`** method converts the file into a string and allows me to read it. I applied the **`.read()`** method to the file variable identified in the with statement. Then, I assigned the string output of this method to the variable **`ip_addresses`**.

In summary, this code reads the contents of the "**`allow_list.txt`**" file into a string format that allows me to later use the string to organize and extract data in my Python program.

Convert the string into a list

In order to remove individual IP addresses from the allow list, I needed it to be in list format. Therefore, I next used the **`.split()`** method to convert the **`ip_addresses`** string into a list:

```
# Use `.split()` to convert `ip_addresses` from a string to a list

ip_addresses = ip_addresses.split()
```

The **`.split()`** function is called by appending it to a string variable. It works by converting the contents of a string to a list. The purpose of splitting **`ip_addresses`** into a list is to make it easier to remove IP addresses from the allow list. By default, the **`.split()`** function splits the text by whitespace into list elements. In this algorithm, the **`.split()`** function takes the data stored in the variable **`ip_addresses`**, which is a string of IP addresses that are each separated by a whitespace, and it converts this string into a list of IP addresses. To store this list, I reassigned it back to the variable **`ip_addresses`**.

Iterate through the remove list

A key part of my algorithm involves iterating through the IP addresses that are elements in the **`remove_list`**. To do this, I incorporated a for loop:

```
# Build iterative statement
# Name loop variable `element`
# Loop through `remove_list`

for element in remove_list:
```

The for loop in Python repeats code for a specified sequence. The overall purpose of the for loop in a Python algorithm like this is to apply specific code statements to all elements in a sequence. The **`for`** keyword starts the for loop. It is followed by the loop variable element and the keyword in. The keyword in indicates to iterate through the sequence **`ip_addresses`** and assign each value to the loop variable element.

Remove IP addresses that are on the remove list

My algorithm requires removing any IP address from the allow list, **`ip_addresses`**, that is also contained in **`remove_list.`** Because there were not any duplicates in **`ip_addresses`**, I was able to use the following code to do this:

```
for element in remove_list:

  # Create conditional statement to evaluate if `element` is in `ip_addresses`

    if element in ip_addresses:

      # use the `.remove()` method to remove
      # elements from `ip_addresses`

        ip_addresses.remove(element)
```

First, within my **`for`** loop, I created a conditional that evaluated whether or not the loop variable element was found in the **`ip_addresses`** list. I did this because applying **`.remove()`** to elements that were not found in **`ip_addresses`** would result in an error.

Then, within that conditional, I applied **`.remove()`** to **`ip_addresses`**. I passed in the loop variable element as the argument so that each IP address that was in the **`remove_list`** would be removed from **`ip_addresses`**.

Update the file with the revised list of IP addresses

As a final step in my algorithm, I needed to update the allow list file with the revised list of IP addresses. To do so, I first needed to convert the list back into a string. I used the **`.join()`** method for this:

```
# Convert `ip_addresses` back to a string so that it can be written into the text file

ip_addresses = "\n".join(ip_addresses)
```

The **`.join()`** method combines all items in an iterable into a string. The **`.join()`** method is applied to a string containing characters that will separate the elements in the iterable once joined into a string. In this algorithm, I used the **`.join()`** method to create a string from the list **`ip_addresses`** so that I could pass it in as an argument to the **`.write()`** method when writing to the file "**`allow_list.txt`**". I used the string ("**`\n`**") as the separator to instruct Python to place each element on a new line.

Then, I used another with statement and the **`.write()`** method to update the file:

```
  # Build `with` statement to rewrite the original file

  with open(import_file, "w") as file:

    # Rewrite the file, replacing its contents with `ip_addresses`

    file.write(ip_addresses)
```

This time, I used a second argument of "**`w`**" with the **`open()`** function in **with** statement. This argument indicates that I want to open a file to write over its contents. When using this argument "**`w`**", I can call the **`.write()`** function in the body of the with statement. The **`.write()`** function writes string data to a specified file and replaces any existing file content.

In this case I wanted to write the updated allow list as a string to the file "**`allow_list.txt`**". This way, the restricted content will no longer be accessible to any IP addresses that were removed from the allow list. To rewrite the file, I appended the **`.write()`** function to the file object file that I identified in the with statement. I passed in the **`ip_addresses`** variable as the argument to specify that the contents of the file specified in the with statement should be replaced with the data in this variable.

Summary

I created an algorithm that removes IP addresses identified in a **`remove_list`** variable from the "**`allow_list.txt`**" file of approved IP addresses. This algorithm involved opening the file, converting it to a string to be read, and then converting this string to a list stored in the variable **`ip_addresses`**. I then iterated through the IP addresses in **`remove_list`**. With each iteration, I evaluated if the element

was part of the **`ip_addresses`** list. If it was, I applied the **`.remove()`** method to it to remove the element from **`ip_addresses`**. After this, I used the .**`join()`** method to convert the **`ip_addresses`** back into a string so that I could write over the contents of the "**`allow_list.txt`**" file with the revised list of IP addresses.